

AI FAQ ASSISTANT API	
TEAM ID: SWTID-2026-5919	TEAM SIZE: 5
TEAM LEADER:	HARINI K
TEAM MEMBER:	AKSHAYAPRIYA M
TEAM MEMBER:	AMREEN FATHIMA M
TEAM MEMBER:	ILAKIYA S
TEAM MEMBER:	DEVAMITHRA C A

1. Project Description

The AI FAQ Assistant API is an intelligent backend application developed to simplify the creation, organization, and retrieval of frequently asked questions. It provides a centralized platform where users can manage FAQ content and obtain AI-generated answers for different topics.

The application is developed using Node.js and Express.js to provide RESTful API services. MongoDB is used to store user accounts and FAQ records, while Mongoose provides schema definition, data validation, and database interaction.

The system integrates Google Gemini AI to generate relevant questions and answers based on user-provided topics. It also provides search functionality that allows users to retrieve information from the available FAQ collection.

Security is implemented through JSON Web Token (JWT) authentication, password hashing using bcrypt, and protected API routes. The application follows a modular backend structure that separates routing, business logic, authentication, and database operations.

The primary objective of this project is to reduce repetitive manual support activities and provide a reliable, reusable API for intelligent FAQ management.

2. Scenario-Based Case Study

2.1 Background

Consider a company that receives a large number of customer inquiries regarding its products, services, account management, and technical issues.

The customer support team frequently answers the same questions. As the number of customers increases, maintaining accurate FAQ documentation and responding to repeated queries becomes increasingly difficult.

The organization requires a centralized system that can organize existing answers, generate new FAQ content, and allow customers to find relevant information quickly.

2.2 Problem Statement

- Repeated customer questions increase the workload of support employees.
- Preparing and updating FAQ documents requires considerable time and effort.

- Unorganized FAQ records make information difficult to locate.
- Manual classification of questions can lead to inconsistent categories.
- Customers may experience delays while waiting for answers to common queries.
- Unauthorized modifications can compromise the accuracy of FAQ content.
- Maintaining consistent and reusable support information becomes difficult as the database grows.

2.3 Proposed Solution

The AI FAQ Assistant API provides an automated backend solution for managing FAQ information and supporting customer queries.

The system enables authorized users to create and maintain FAQ records, while AI integration helps generate question-and-answer content from specified topics.

A structured search mechanism retrieves relevant FAQ entries using user-provided keywords. Authentication middleware restricts access to protected operations, and MongoDB stores the application’s persistent data.

The API can be integrated with websites, mobile applications, or other customer support interfaces without requiring the business logic to be rewritten for each platform.

2.4 Application Usage

User Type	Typical Activities
Content Creator	Registers and logs in; creates FAQ entries manually; generates FAQ content using AI; updates or deletes personally created FAQs; organizes questions into categories.
Customer / Public User	Views available FAQ information; searches for answers using keywords; retrieves relevant support content through API requests.
Administrator	Manages user accounts and FAQ records; maintains FAQ categories; monitors stored content; oversees access to administrative operations.

2.5 Expected Outcome

The proposed system reduces the effort required to prepare and maintain frequently asked questions. It makes support information easier to access and provides a consistent method for managing FAQ records.

The combination of AI-assisted content generation, structured storage, and secure API access creates a foundation for scalable customer support automation.

3. Software Requirements

Software / Technology	Purpose
Operating System	Windows 10/11, macOS, or Linux
Node.js	Version 18 or above; executes backend JavaScript
npm	Installs and manages project dependencies
Express.js	Creates API routes and handles HTTP requests
MongoDB	Stores users and FAQ documents
Mongoose	Defines schemas and communicates with MongoDB
Google Gemini API	Generates AI-based FAQ content
JSON Web Token	Provides token-based authentication

Software / Technology	Purpose
bcrypt	Hashes passwords before database storage
Postman / Thunder Client	Tests API endpoints and responses
Visual Studio Code	Development and debugging environment
dotenv	Loads configuration values from environment variables

4. Hardware Requirements

The application can run on a standard development computer.

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 or equivalent	Intel Core i5 / Ryzen 5
RAM	4 GB	8 GB or more
Storage	1 GB free space	5 GB free space
Network	Internet connection	Stable broadband connection

An internet connection is required for accessing a cloud-hosted database or the Google Gemini API, if those services are used.

5. Project Architecture

The AI FAQ Assistant API follows a layered backend architecture. Each layer performs a specific responsibility, making the application easier to maintain, test, and extend.

5.1 Architecture Components

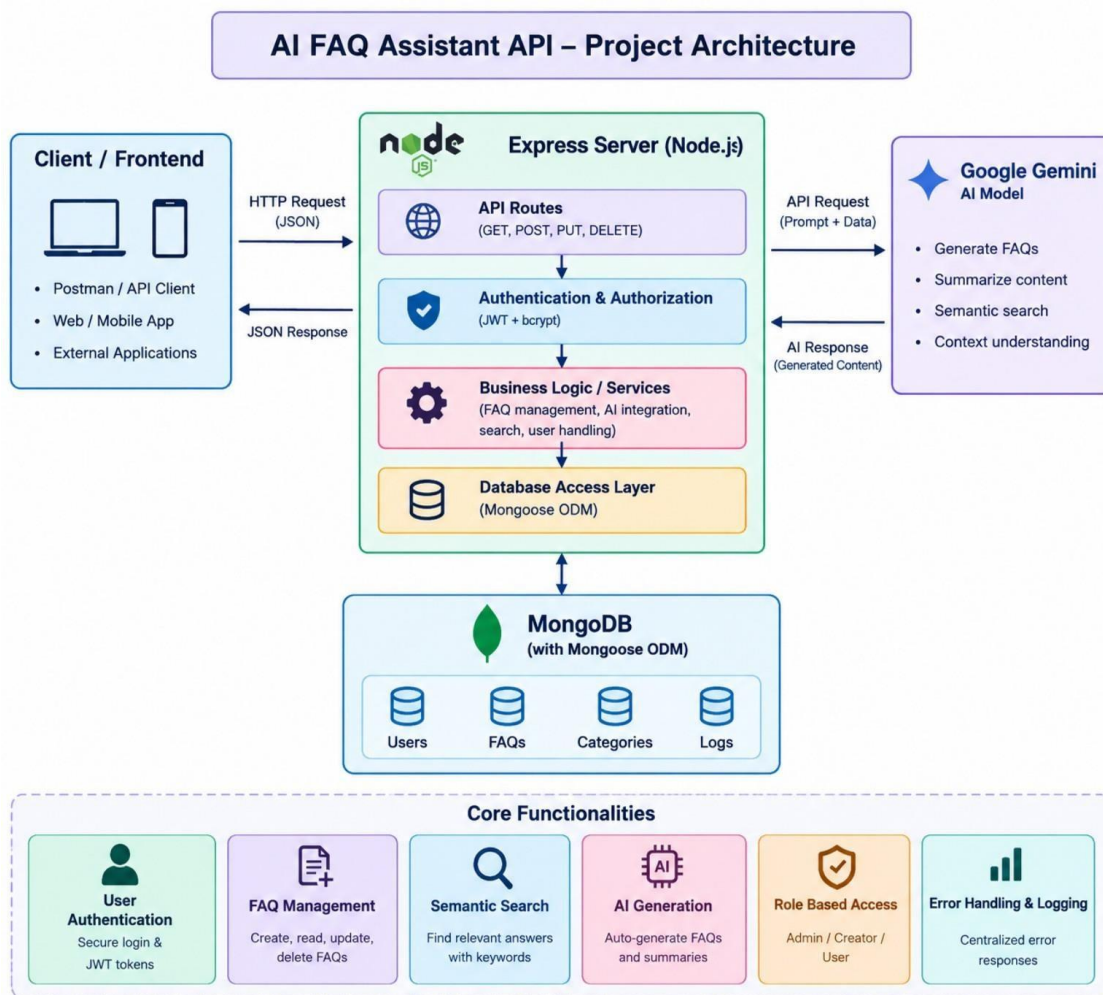


Figure 5.1 – AI FAQ Assistant API Project Architecture

5.2 Component Definitions

1. Client Layer

The client sends HTTP requests to the backend and receives JSON responses. Postman or Thunder Client can be used to test the application without developing a separate frontend.

2. Express.js Server

The Express server receives incoming requests, identifies the requested route, parses JSON data, and forwards the request to the appropriate handler.

3. Authentication Middleware

This layer checks JWT tokens for protected endpoints. It verifies the user's identity and ensures that restricted operations are available only to authorized users.

4. Controller Layer

Controllers process incoming requests, apply application rules, interact with database models or AI services, and generate suitable API responses.

5. Database Layer

MongoDB stores user information and FAQ records. Mongoose defines the structure of these records and performs database operations.

6. AI Integration Layer

The AI service communicates with Google Gemini to generate FAQ questions, answers, and related content based on the supplied topic.

7. Error Handling Layer

A centralized error handler manages unexpected failures and returns consistent error responses without exposing sensitive internal information.

6. Database Design and ER Diagram

The database is organized around user accounts and FAQ records. Each FAQ is associated with the user who created it.

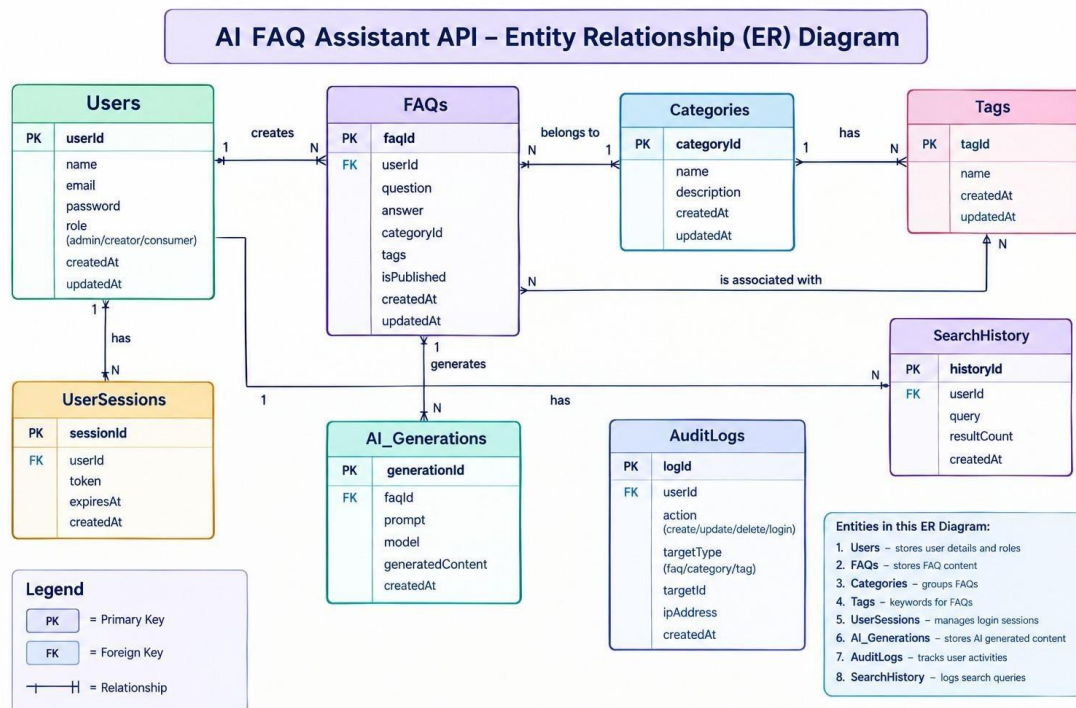


Figure 6.1 – AI FAQ Assistant API Entity Relationship (ER) Diagram

6.1 User Entity

Attribute	Description
<code>_id</code>	Unique identifier for each user
<code>name</code>	Name of the registered user
<code>email</code>	Unique email address
<code>password</code>	Hashed password
<code>role</code>	User role, such as user or admin
<code>createdAt</code>	Account creation timestamp

Attribute	Description
updatedAt	Last account update timestamp

6.2 FAQ Entity

Attribute	Description
_id	Unique identifier for the FAQ
question	Frequently asked question
answer	Corresponding answer
category	Topic or classification
createdBy	Reference to the creator's user ID
createdAt	FAQ creation timestamp
updatedAt	Last modification timestamp

6.3 AI-Generated FAQ Data

The following fields represent the content generated by the AI service before it is saved as a regular FAQ record.

Attribute	Description
topic	Topic supplied to the AI
generatedQuestion	AI-generated question
generatedAnswer	AI-generated answer
generatedCategory	Suggested FAQ category
generatedAt	Time of generation

These fields can be maintained as temporary service output or persisted separately if the implementation requires a dedicated generation history.

6.4 Category Entity

Attribute	Description
categoryName	Name of the category
description	Details about the category
totalFAQs	Number of FAQs associated with it

The category information may be stored as a separate collection or represented through the category field in each FAQ document, depending on the implemented database design.

6.5 Entity Relationships

- **User to FAQ — One-to-Many:** One user can create multiple FAQ records.
- **Category to FAQ — One-to-Many:** A category can contain multiple FAQ entries.
- **User to AI Generation — One-to-Many:** A user can request multiple AI-generated FAQ responses.
- **FAQ to Search Results — Logical Relationship:** Search requests return FAQ records matching the supplied query.

7. Key Features

7.1 User Authentication

The application supports user registration and login. Passwords are hashed using bcrypt, and JWT tokens are used to authenticate requests to protected routes.

7.2 AI-Based FAQ Generation

Users can provide a topic or subject to the AI service. The system generates relevant FAQ questions and answers that can be reviewed and saved.

7.3 FAQ Management

The API supports creating, retrieving, updating, and deleting FAQ records through standard REST operations.

7.4 FAQ Search

Users can search the available FAQ collection using keywords. The API returns matching entries to help users locate relevant information.

7.5 Category-Based Organization

FAQs are grouped according to their subject or category. This allows related information to be maintained in a structured manner.

7.6 Role-Based Access Control

Access permissions can be applied according to the user's role. Administrative functions are separated from ordinary content management operations.

7.7 Data Validation

Mongoose schema rules validate required fields and data formats before records are stored in the database.

7.8 Centralized Error Handling

Errors are processed through a common middleware component, providing consistent status codes and readable error messages.

7.9 Modular REST API

The backend is divided into separate route, controller, model, and middleware modules. This structure supports easier debugging and future development.

7.10 Secure Configuration

Environment variables are used to store sensitive configuration values, such as database connection strings, JWT secrets, and AI service credentials.

8. Roles and Responsibilities

8.1 Administrator

- Manage registered user accounts.

- View and manage FAQ records.
- Maintain FAQ categories.
- Review AI-generated content.
- Perform authorized administrative operations.
- Monitor the overall content management process.

8.2 Content Creator

- Create new FAQ entries.
- Generate FAQ content using AI.
- Edit personally created FAQs.
- Delete FAQs they own.
- Assign appropriate categories to FAQ records.

8.3 Authenticated User

- Access the personal profile.
- View published FAQ content.
- Search for relevant answers.
- Use AI-assisted FAQ generation where permitted.

8.4 Public User

- View publicly accessible FAQs.
- Search for available information.
- Access permitted public API endpoints.
- Cannot modify FAQ records or access protected resources.

9. User Workflow

Public users can directly access the permitted FAQ listing and search endpoints without completing the login process. Authenticated content creators first register or log in, receive a JWT token, and then use protected FAQ and AI-generation operations.

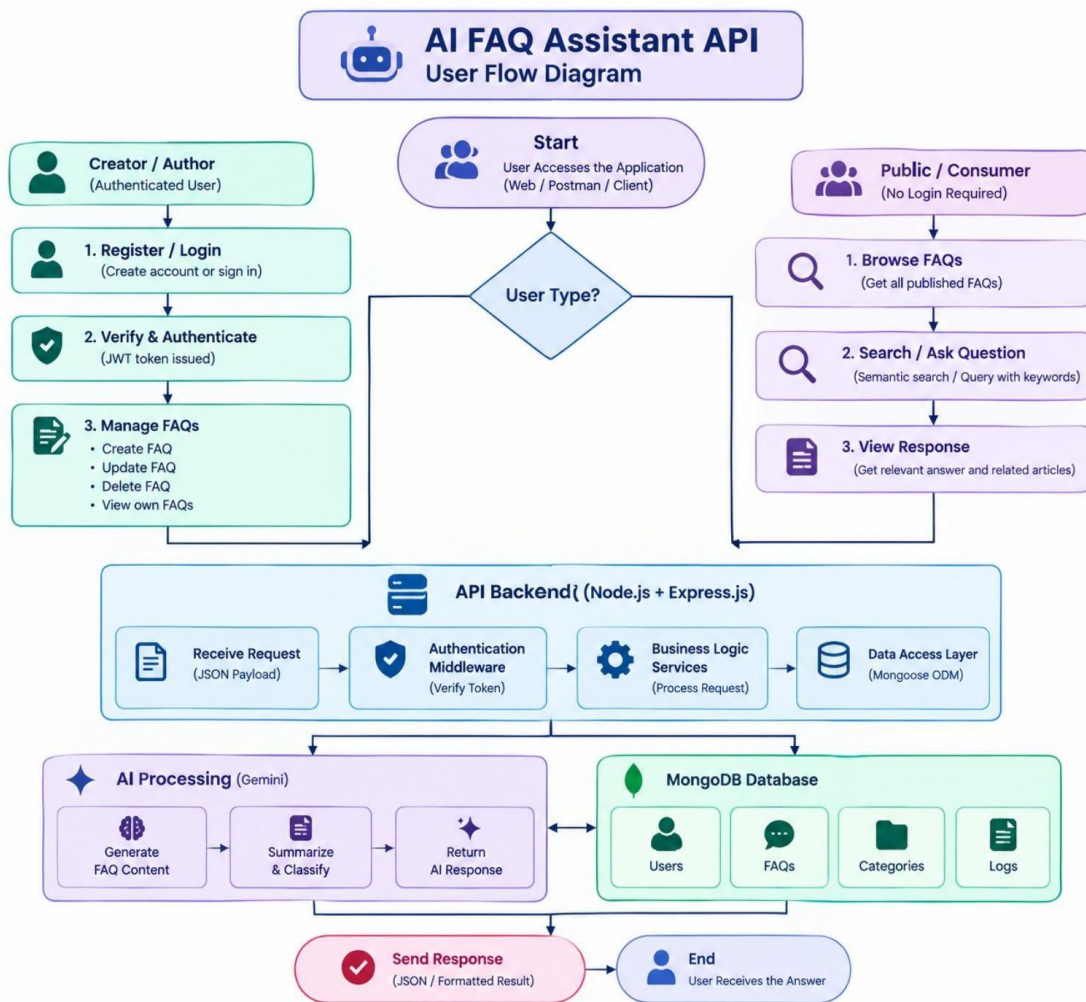


Figure 9.1 – AI FAQ Assistant API User Flow Diagram

10. MVC Architecture Pattern

The project uses a Model-View-Controller-inspired structure to separate the application's data, request processing, and API interface.

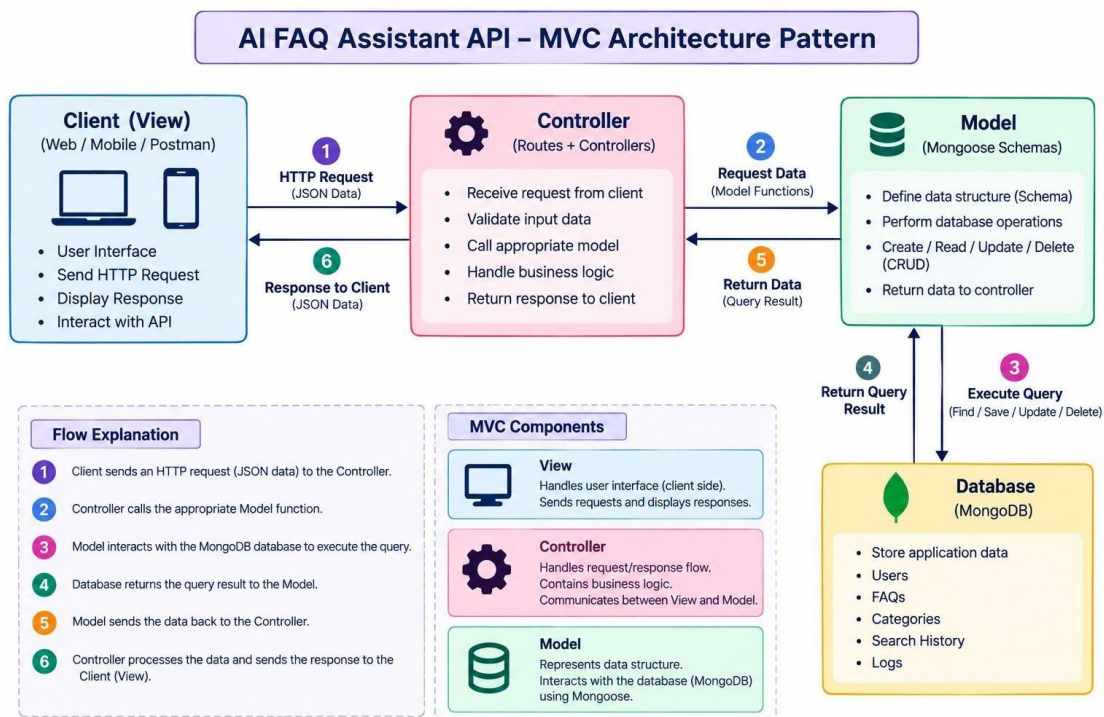


Figure 10.1 – AI FAQ Assistant API MVC Architecture Pattern

10.1 Model Layer

The model layer defines the structure of the data stored in MongoDB. Mongoose schemas specify the fields, data types, validation rules, and relationships used by the application.

10.2 View Layer

The view represents the client side of the system, such as a web application, mobile application, or Postman/Thunder Client interface. It sends HTTP requests and displays the JSON responses returned by the API.

10.3 Controller Layer

The controller layer receives requests, validates input, applies business logic, calls model or service functions, and prepares the response returned to the client.

10.4 Routing Layer

The routing layer maps HTTP methods and URL paths to their corresponding controller functions. Authentication middleware is applied to routes that require a valid user session.

11. Project Setup and Configuration

11.1 Creating the Project

1. Create a folder named ai-faq-assistant-api.
2. Open the folder in Visual Studio Code.
3. Initialize a Node.js project.
4. Install the required dependencies.
5. Create the source folders and configuration files.

11.2 Initializing the Application

Run the following commands in the project terminal:

```
mkdir ai-faq-assistant-api
cd ai-faq-assistant-api
npm init -y
```

Install the required packages:

```
npm install express mongoose dotenv cors jsonwebtoken bcrypt @google/genai
```

For development, install a tool to restart the server automatically:

```
npm install --save-dev nodemon
```

11.3 Suggested Folder Structure

```
ai-faq-assistant-api/
├── src/
│   ├── config/
│   │   └── db.js
│   ├── controllers/
│   │   ├── authController.js
│   │   ├── faqController.js
│   │   └── aiController.js
│   ├── middleware/
│   │   ├── authMiddleware.js
│   │   └── errorMiddleware.js
│   ├── models/
│   │   ├── User.js
│   │   └── FAQ.js
│   ├── routes/
│   │   ├── authRoutes.js
│   │   ├── faqRoutes.js
│   │   └── aiRoutes.js
│   ├── services/
│   │   └── aiService.js
│   ├── app.js
│   └── server.js
├── .env
├── .gitignore
├── package.json
└── README.md
```

11.4 Environment Configuration

Create a .env file in the project root:

```
PORT=5000
MONGODB_URI=your_mongodb_connection_string
```

```
JWT_SECRET=your_jwt_secret  
GEMINI_API_KEY=your_gemini_api_key
```

Do not upload actual API keys, passwords, or secret values to a public repository.

12. Backend Development

12.1 Server Configuration

The server entry point initializes the Express application, connects to MongoDB, and starts listening for incoming requests.

File: src/server.js

Responsibilities:

- Load environment variables.
- Establish the database connection.
- Start the HTTP server.
- Handle server startup errors.

12.2 Express Application Setup

File: src/app.js

The Express application configures middleware and registers API routes.

Responsibilities:

- Enable JSON request parsing.
- Configure CORS.
- Register authentication, FAQ, and AI routes.
- Attach centralized error-handling middleware.

12.3 Authentication Middleware

File: src/middleware/authMiddleware.js

This middleware reads the JWT token supplied in the request's Authorization header. It verifies the token and makes the authenticated user information available to protected route handlers.

Requests with missing or invalid credentials are rejected.

12.4 Error Handling Middleware

File: src/middleware/errorMiddleware.js

The error middleware captures errors raised during request processing and returns a standardized JSON response.

Example error response:

```
{  
  "success": false,  
  "message": "An unexpected error occurred"  
}
```

The actual response should use the appropriate HTTP status code and avoid exposing internal stack traces or sensitive configuration details.

13. Database Configuration

13.1 User Schema

The User model represents registered users. It stores essential account details and applies validation rules to required fields.

The password is hashed before it is stored. The email field is configured to support unique user identification.

13.2 FAQ Schema

The FAQ model stores the question, answer, category, and creator reference.

The schema validates the required content and maintains timestamps to record when an FAQ was created or modified.

13.3 Database Operations

Mongoose provides methods for the following operations:

- Insert new user and FAQ documents.
- Retrieve records by ID or search criteria.
- Update existing FAQ documents.
- Delete authorized records.
- Validate input against the defined schema.

14. API Endpoint Specification

The API uses standard HTTP methods to perform authentication, FAQ management, search, and AI generation.

14.1 Authentication APIs

Method	Endpoint	Purpose	Access
POST	/api/auth/register	Register a new user	Public
POST	/api/auth/login	Authenticate a user	Public

14.2 FAQ APIs

Method	Endpoint	Purpose	Access
GET	/api/faqs	Retrieve FAQ records	Public
GET	/api/faqs/search?q=keyword	Search FAQs	Public
GET	/api/faqs/:id	Retrieve a specific FAQ	As configured
POST	/api/faqs	Create a new FAQ	Protected
PUT	/api/faqs/:id	Update an FAQ	Protected
DELETE	/api/faqs/:id	Delete an FAQ	Protected

14.3 AI APIs

Method	Endpoint	Purpose	Access
POST	/api/ai/generate-faq	Generate an FAQ from a topic	Protected

Endpoint names and access permissions should be verified against the actual route files before submission.

15. API Testing and Execution

The application can be tested using Postman or Thunder Client.

15.1 Starting the Server

Add the following development script to package.json if it is not already present:

```
{
  "scripts": {
    "dev": "nodemon src/server.js"
  }
}
```

Run the application:

```
npm run dev
```

If the server starts successfully, it should connect to the configured database and begin listening on the specified port.

15.2 Test Case 1: User Registration

POST /api/auth/register

Request:

```
{
  "name": "Arun Kumar",
  "email": "arun@example.com",
  "password": "SecurePass123"
}
```

Expected result: A new user account is created after successful validation. The password is hashed before storage.

15.3 Test Case 2: User Login

POST /api/auth/login

Request:

```
{
  "email": "arun@example.com",
  "password": "SecurePass123"
}
```

Expected result: The credentials are verified, and a JWT token is returned if authentication succeeds.

15.4 Test Case 3: Create FAQ

POST /api/faqs

Request:

```
Authorization: Bearer <JWT_TOKEN>
{
  "question": "How can I reset my account password?",
}
```

```
"answer": "Open the account settings, select the password reset option, and follow the instructions.",
"category": "Account Management"
}
```

Expected result: The FAQ is validated and saved with a reference to the authenticated creator.

15.5 Test Case 4: Search FAQ

GET /api/faqs/search?q=password

Expected result: The API returns FAQ records matching the search term, along with the relevant response data.

15.6 Test Case 5: Generate FAQ Using AI

POST /api/ai/generate-faq

Request:

```
Authorization: Bearer <JWT_TOKEN>
{
  "topic": "Online payment security"
}
```

Expected result: The backend sends the topic to the AI service and returns a generated FAQ question and answer. The generated content can then be reviewed and saved.

15.7 Additional Test Cases

Test Scenario	Expected Behavior
Register with missing fields	Validation error
Register with an existing email	Duplicate account rejected
Login with incorrect password	Authentication error
Access protected route without token	Unauthorized response
Create FAQ with missing question	Validation error
Search using a matching keyword	Matching FAQs returned
Update an FAQ without permission	Access denied
Request an unknown endpoint	Not-found response
AI service failure	Appropriate error response

16. Security Considerations

Security is an important aspect of the AI FAQ Assistant API.

- **Password protection:** Store password hashes rather than plain-text passwords.
- **Token verification:** Validate JWT tokens before granting access to protected resources.
- **Authorization checks:** Verify that a user has permission to modify or delete the requested FAQ.
- **Input validation:** Validate request fields before processing or saving data.
- **Environment variables:** Keep secrets and credentials outside source code.
- **Error sanitization:** Avoid returning database details, stack traces, or secret values to clients.
- **AI output review:** Review AI-generated answers for accuracy before publishing them.

- **API configuration:** Apply suitable request limits and access controls when deploying the service.

17. Advantages of the Proposed System

- Reduces repetitive manual FAQ preparation.
- Supports automatic generation of question-and-answer content.
- Provides a centralized repository for support information.
- Improves the accessibility of stored FAQ records.
- Enables structured categorization and keyword search.
- Protects restricted operations through authentication.
- Separates application logic into reusable modules.
- Supports integration with different frontend applications.
- Simplifies the maintenance of FAQ records.
- Provides a foundation for extending customer support automation.

Conclusion

The AI FAQ Assistant API combines a modular Node.js and Express.js backend, MongoDB data storage, secure authentication, structured FAQ management, search capabilities, and Google Gemini-based content generation. The architecture and MVC-inspired organization provide a clear separation of responsibilities, making the system suitable for development, testing, and future extension.